

Failure (a real rollback trigger) — Solution

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Story — `rollback_trigger_practice.py`: a rollback plan that has never fired is only an idea. This file plays back a score sequence that really drops, and checks the trigger fires at the right moment — and only once. **If not:** the first time your rollback ran would be during a real incident.

Every example below plays back this same simulated score sequence — a healthy start, then a sustained drop partway through, acting out the exact situation this exercise asks for:

```
# rollback_trigger_practice.py
scores = [0.95, 0.94, 0.96, 0.93, 0.95, 0.60, 0.55, 0.58, 0.52, 0.50]
baseline = 0.95
```

Basic Version

Approach 1 — the direct way

Story: average the last few scores, compare with the baseline, roll back if the drop is too big. **If not:** a slow, real decline would never trigger anything.

```
# rollback_trigger_practice.py
import logging

logging.basicConfig(level=logging.INFO)

def should_rollback(scores_so_far, baseline, window_size=5, drop_threshold=0.2):
    if len(scores_so_far) < window_size:
        return False
    recent = scores_so_far[-window_size:]
    avg_recent = sum(recent) / len(recent)
    percent_drop = (baseline - avg_recent) / baseline
    return percent_drop >= drop_threshold

def rollback_to_previous():
    logging.critical("ROLLBACK: reverting to last known-good version")
    return "v1"
```

```

scores = [0.95, 0.94, 0.96, 0.93, 0.95, 0.60, 0.55, 0.58, 0.52, 0.50]
seen = []
for i, s in enumerate(scores):
    seen.append(s)
    if should_rollback(seen, baseline=0.95):
        print(f"request {i}: score {s} -- triggering rollback")
        rollback_to_previous()

```

Expected output:

```

request 7: score 0.58 -- triggering rollback
CRITICAL:root:ROLLBACK: reverting to last known-good version
request 8: score 0.52 -- triggering rollback
CRITICAL:root:ROLLBACK: reverting to last known-good version
request 9: score 0.5 -- triggering rollback
CRITICAL:root:ROLLBACK: reverting to last known-good version

```

It correctly waits until request 7 (the 8th score) before firing — that's the first point where the 5 most recent scores ([0.93, 0.95, 0.60, 0.55, 0.58] , averaging 0.722) are far enough below the 0.95 baseline (a 24% drop) to clear the 20% threshold. This version works correctly for what the exercise asks, but it calls `rollback_to_previous()` three separate times for what is really one decline — that's the flapping problem, fixed below.

Revision from Doc01 (Basic): the rollback message uses `logging.critical(...)`, not `print(...)`. A rollback means the live version is failing users right now — Doc01's definition of `CRITICAL`: "a failure serious enough that the whole program can't keep going as it is." A `print` would vanish with the terminal; a log line can be kept and searched later.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — type hints, named parameters, same repeat-firing behavior

Story: the terminal should stay quiet, but after an incident you want every window check on record. Two log levels — `INFO` on screen, `DEBUG` in a file — give you both. **If not:** "how fast did the score fall?" would have no answer after the fact.

```
# rollback_trigger_practice.py
import logging

logger = logging.getLogger("rollback")
logger.setLevel(logging.DEBUG)
console = logging.StreamHandler()
console.setLevel(logging.INFO)
logfile = logging.FileHandler("rollback.log")
logfile.setLevel(logging.DEBUG)
logger.addHandler(console)
logger.addHandler(logfile)

def should_rollback(
    scores_so_far: list[float],
    baseline: float,
    window_size: int = 5,
    drop_threshold: float = 0.2,
) -> bool:
    # when: not enough scores yet for a fair average – never fire early
    if len(scores_so_far) < window_size:
        return False
    # how: only the most recent window counts
    recent = scores_so_far[-window_size:]
    avg_recent = sum(recent) / len(recent)
    percent_drop = (baseline - avg_recent) / baseline
    logger.debug("window avg=%.3f drop=%.1f%%", avg_recent, percent_drop * 100)
    return percent_drop >= drop_threshold

def rollback_to_previous() -> str:
    logger.critical("ROLLBACK: reverting to last known-good version")
    return "v1"

def watch_and_rollback(scores: list[float], baseline: float) -> None:
```

```

seen: list[float] = []
for i, s in enumerate(scores):
    seen.append(s)
    if should_rollback(seen, baseline):
        print(f"request {i}: score {s} -- triggering rollback")
        rollback_to_previous()

scores = [0.95, 0.94, 0.96, 0.93, 0.95, 0.60, 0.55, 0.58, 0.52, 0.50]
watch_and_rollback(scores, baseline=0.95)

```

Expected output: the same firing pattern as Basic's — fires at request 7, and again at 8 and 9, since `should_rollback` still has no memory between calls. The terminal shows only the `request N` lines and the `ROLLBACK` messages; `rollback.log` also holds a `DEBUG` line for every window check, like `window avg=0.722 drop=24.0%`.

Revision from Doc01 (Intermediate): this is Doc01's "logger with two output levels" exercise, reused as-is: the terminal stays quiet (`INFO` and above), while the file keeps every detail (`DEBUG` and above). After a real rollback, the first question is always "how fast did the score fall?" — the `DEBUG` lines in `rollback.log` answer it, without filling the terminal during normal operation.

Difference from Basic: Approach 1 adds full type hints and separates the watching loop into its own `watch_and_rollback` function, but the core check is unchanged, so it still flaps exactly the way Basic does. That's intentional — it shows the repeat firing is about *memory*: the loop has to remember it already rolled back. Approach 2 adds exactly that.

Approach 2 — fire once, then stop watching

Story: one decline should cause one rollback, not one per request. A simple flag, set the moment the rollback fires, makes the loop remember it already acted. **If not:** a falling score would roll back again and again, each time to the same version.

```

# rollback_trigger_practice.py
def watch_and_rollback_once(scores: list[float], baseline: float) -> None:
    seen: list[float] = []
    rolled_back = False
    for i, s in enumerate(scores):
        seen.append(s)
        # why: after the first rollback there is nothing more to do –
        # the old version is live again
        if rolled_back:
            continue
        if should_rollback(seen, baseline):
            print(f"request {i}: score {s} -- triggering rollback")
            rollback_to_previous()
            rolled_back = True

```

```
scores = [0.95, 0.94, 0.96, 0.93, 0.95, 0.60, 0.55, 0.58, 0.52, 0.50]
watch_and_rollback_once(scores, baseline=0.95)
```

Expected output:

```
request 7: score 0.58 -- triggering rollback
ROLLBACK: reverting to last known-good version
```

(the second line is the `CRITICAL` log from Approach 1's logger, which has no formatter.) Exactly one rollback, at request 7 — later requests are still recorded, but nothing fires again.

Which one should you actually write? Approach 2 — the moment this runs against real, continuous traffic, it must remember that it already acted. Once a *fixed* version is deployed, start a fresh watch (a new `seen` list and flag) so the next decline can be caught too.